Get Certified
Document your skills with W3Schools Certificates
~~$1,995~~ $699
Save 65% now!

JavaScript Iterables

Previous Next

An Iterable is an Iterable Object

An iterable can be iterated over with `for..of` loops

The For Of Loop

The JavaScript `for..of` statement loops through the elements of an iterable object.

Syntax

```
for (variable of iterable) {
  // code block to be executed
}
```

Iterating

Iterating means looping over a sequence of elements.

Here are some easy examples:

- Iterating over a String
- Iterating over an Array
- Iterating over a Set
- Iterating over a Map

Example

```
const name = "W3Schools";

for (const x of name) {
  // code block to be executed
}
```

Try it Yourself »

Iterating Over an Array

You can use a `for..of` loop to iterate over the elements of an Array:

Example 1

```
const letters = ["a","b","c"];

for (const x of letters) {
  // code block to be executed
}
```

Try it Yourself »

Example 2

```
const numbers = [2,4,6,8];

for (const x of numbers) {
  // code block to be executed
}
```

Try it Yourself »

Example

```
const letters = new Set(["a","b","c"]);

for (const x of letters) {
  // code block to be executed
}
```

Try it Yourself »

Iterating Over a Map

You can use a `for..of` loop to iterate over the elements of a Map:

Example

```
const fruits = new Map([
  ["apples", 500],
  ["bananas", 300],
  ["oranges", 200]
]);

for (const x of fruits) {
  // code block to be executed
}
```

Try it Yourself »

ADVERTISEMENT
Get Certified Document your skills with W3Schools Certificates ~~$1,995~~ $699 Save 65%

JavaScript Iterators

The **iterator protocol** defines how to produce a **sequence of values** from an object.

An object becomes an **iterator** when it implements a `next()` method.

The `next()` method must return an object with two properties:

- value (the next value)
- done (true or false)

value	The value returned by the iterator (Can be omitted if done is true)
done	*true* if the iterator has completed *false* if the iterator has produced a new value

Note

Technically, iterables must implement the `Symbol.iterator` method.

In JavaScript the following are iterables:

- Strings
- Arrays
- Typed Arrays
- Sets
- Maps

Because their prototype objects have a `Symbol.iterator` method:

Home Made Iterable

This iterable returns never ending number: 10,20,30,40,.... when `the next()` method is called:

Example

```
// Home Made Iterable
function myNumbers() {
```

```
      return {value:n, done:false};
    }
  };
}

// Run the Iterable
const n = myNumbers();
n.next(); // Returns 10
n.next(); // Returns 20
n.next(); // Returns 30
```

Try it Yourself »

Note

Home made iterables do not support the JavaScript `for..of` statement.

A JavaScript iterable is an object that has a **Symbol.iterator**.

The `Symbol.iterator` is a function that returns a `next()` function.

An iterable can be iterated over with the code: `for (const x of iterable) { }`

Example

```
// Create an Object
myNumbers = {};

// Make it Iterable
myNumbers[Symbol.iterator] = function() {
  let n = 0;
  done = false;
  return {
    next() {
      n += 10;
      if (n == 100) {done = true}
      return {value:n, done:done};
    }
  };
}
```

```
}
```

Try it Yourself »

The Symbol.iterator method is called automatically by `for..of`.

But we can also do it "manually":

Example

```
let iterator = myNumbers[Symbol.iterator]();

while (true) {
  const result = iterator.next();
  if (result.done) break;
  // Any Code Here
}
```

Try it Yourself »

Note

Iterators provide a controlled way to work with **data sequences**, enabling **custom iteration logic** for various data structures.

In the next chapter you will learn about the new **Iterator Helper Functions** introduced in ECMAScript 2025.

Exercise ?

```
  i++;
}
```

What will be the value of `i`?

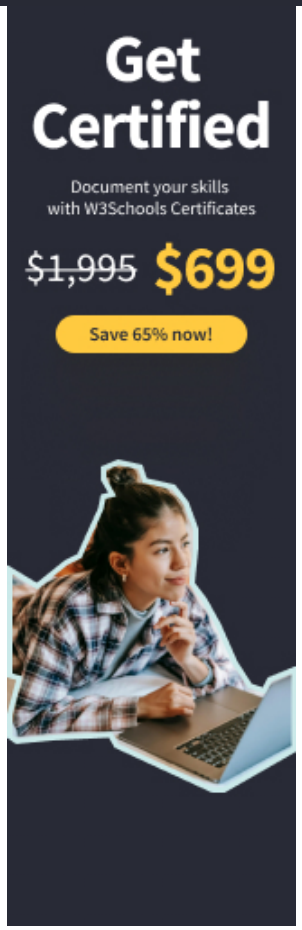
- 0
- 1
- 2
- 3

Submit Answer »

❮ Previous Next ❯

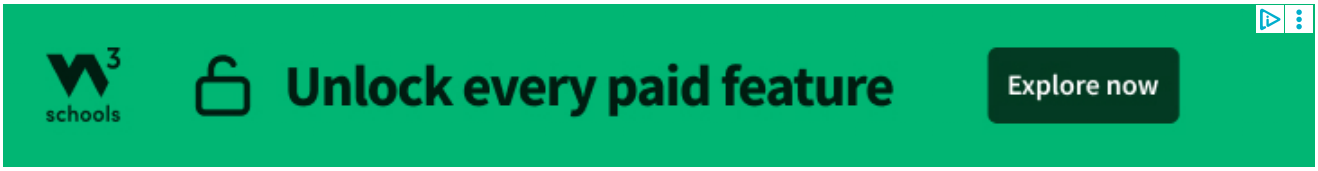
Track your progress - it's free!
Sign Up Log in

ADVERTISEMENT



COLOR PICKER

ADVERTISEMENT



ADVERTISEMENT

Top Tutorials

HTML Tutorial
CSS Tutorial
JavaScript Tutorial
How To Tutorial
SQL Tutorial
Python Tutorial
W3.CSS Tutorial
Bootstrap Tutorial
PHP Tutorial
Java Tutorial
C++ Tutorial
jQuery Tutorial

Top References

HTML Reference
CSS Reference
JavaScript Reference
SQL Reference
Python Reference
W3.CSS Reference
Bootstrap Reference
PHP Reference
HTML Colors
Java Reference
Angular Reference
jQuery Reference

Top Examples

HTML Examples
CSS Examples
JavaScript Examples
How To Examples
SQL Examples
Python Examples
W3.CSS Examples
Bootstrap Examples
PHP Examples
Java Examples
XML Examples
jQuery Examples

Get Certified

HTML Certificate
CSS Certificate
JavaScript Certificate
Front End Certificate
SQL Certificate
Python Certificate
PHP Certificate
jQuery Certificate
Java Certificate
C++ Certificate
C# Certificate
XML Certificate

Copyright 1999-2025 by Refsnes Data. All Rights Reserved. W3Schools is Powered by W3.CSS.